

Parte 1

7	• Se espera utilizar Tabla de Hash o Trie para poder resolver el evento FIND-WORD y necesariamente Trie para los eventos FIND-PREFIX y FIND-SUFFIX. • Ya sea usando Trie o Tabla de Hash, la complejidad del evento FIND-WORD es $O(w)$ para una palabra w. Se puede argumentar que es $O(1)$ considerando que el tamaño de las palabras está acotado (analizando los tests). • Para el evento FIND-PREFIX se puede utilizar el trie llegando al nodo donde termina el prefijo y desde ahí recorrer todos los nodos alcanzables, retornando cada palabra formada al llegar a un nodo terminal. Esto tiene complejidad de $O(k \times L)$ siendo k la cantidad de palabras encontradas y L el largo máximo de las palabras (podría ser el promedio también). • Para el evento FIND-SUFFIX la idea es armar un trie pero que guarde las palabras al revés. Luego cuando busquemos el sufijo, lo damos vuelta y seguimos la misma lógica que con FIND-PREFIX lo que tiene complejidad $O(k \times L)$. Los resultados obtenidos serán las palabras al revés, entonces es necesario darlas vuelta. Finalmente, para entregarlas en orden alfabético, se puede utilizar radixsort, que tiene complejidad $O(k \times L)$, con lo que se cumple lo pedido. Evaluación: • 2 pts por usar estructuras correctas. • 1 pto por justificar evento FIND-WORD • 2 pto por justificar evento FIND-PREFIX • 2 pts por justificar evento FIND-SUFFIX. Puntaje parcial por estructuras correctas, pero sin justificación o justificación incompleta.
---	---

Importante: Se otorgará puntaje completo si se emplea un algoritmo distinto al indicado, siempre que cumpla con la complejidad requerida.

Parte 2

6

- Menciona que se utiliza [rolling-hashing](#) y explica el funcionamiento del algoritmo, en especial el proceso para hashear más rápidamente cada substring.
- Justifica que la complejidad temporal es $O(N + Q \times M)$ ya que el hasheo de todas las substrings toma tiempo $O(N)$ y luego cada consulta toma tiempo $O(M)$ al verificar la substring encontrada (si son posibles las colisiones, es necesario verificar).
- Describe qué estructuras se utilizan para guardar información y qué se guarda. Lo esperable es almacenar los hashes a medida que se van calculando, los valores de las potencias (para calcular los hashes) y los índices por cada substring. Luego se justifica que el costo de memoria es $O(N)$ ya que todas las estructuras que se utilizan tienen tamaño del orden $O(N)$.
- Explica que para manejar las colisiones, debe verificar el string antes de retornar el índice. Además, si colisionan dos o más substrings distintos en el mismo hash, es posible manejarlo usando una lista ligada.

Evaluación:

- 2 pts por explicar estrategia de rolling hashing.
- 1 pto por justificar complejidad temporal.
- 1 pto por mencionar estructuras que almacenan información y su costo de memoria asociado.
- 2 pts por explicar manejo de colisiones.

Puntaje parcial por complejidades/estructuras correctas, pero sin explicación o incompleta.

Parte 3

5	Se esperaba el uso de backtracking para resolver el problema y definir los siguientes elementos: • Variables: Siendo el tablero T una matriz de dimensiones $n \times m$: Las variables se definen como el conjunto $\{T_{ij} \mid 0 \leq i < n, 0 \leq j < m\}$ que corresponde a las casillas del tablero. • Dominio: El conjunto de todas las piezas. • Restricciones: Siendo P_{ij} la pieza asignada a T_{ij} – $P_{i,j}.down == P_{i+1,j}.up$ – $P_{i,j}.up == P_{i-1,j}.down$ – $P_{i,j}.right == P_{i,j+1}.left$ – $P_{i,j}.left == P_{i,j-1}.right$ – $\forall P_{ij}.(P_{ij}.up == negro \implies i = 0 \vee T_{i-1,j} == \#)$ – $\forall P_{ij}.(P_{ij}.down == negro \implies i = n - 1 \vee T_{i+1,j} == \#)$ – $\forall P_{ij}.(P_{ij}.left == negro \implies j = 0 \vee T_{i,j-1} == \#)$ – $\forall P_{ij}.(P_{ij}.right == negro \implies j = m - 1 \vee T_{i,j+1} == \#)$ – $\forall T_{ij} == \# \implies$ no hay pieza asignada en T_{ij} Evaluación: • 1 pt por definir variables. • 1 pt por definir dominio. • 3 pts por definir restricciones Corresponde puntaje parcial por sólo explicar con palabras o errores menores.
2	• Una heurística es ordenar las piezas según su peso. Luego, si ya se han encontrado soluciones anteriores, se puede detener el algoritmo si se ve que las piezas disponibles actualmente no podrían sumar suficiente como para alcanzar un puntaje más alto que el ya obtenido. • Una poda consiste en sólo intentar colocar piezas negras en los bordes del tablero o en casillas colindantes a una bloqueada. Evaluación: • 2 pts por ambas optimizaciones (No necesariamente las mencionadas en la pauta). Corresponde puntaje parcial por mencionar sólo una o por mencionar ambas pero no explicarlas.